

DimMem: Dimensional Structuring for Efficient Long-Term Agent Memory

Wentao Qiu^{1,2} Haotian Hu^{1*} Fanyi Wang^{1*} Jinwei Kong^{1,3} Yu Zhang¹

¹StepOS ²Xiamen University ³ShanghaiTech University
wtqiu@stu.xmu.edu.cn
{huhaotian, wangfanyi}@stepos.com

Abstract

Large language model (LLM) agents require long-term memory to leverage information from past interactions. However, existing memory systems often face a fidelity–efficiency trade-off: raw dialogue histories are expensive, while flat facts or summaries may discard the structure needed for precise recall. We propose **DimMem**, a lightweight dimensional memory framework that represents each memory as an atomic, typed, and self-contained unit with explicit fields such as time, location, reason, purpose, and keywords. This representation exposes the structure needed for dimension-aware retrieval, memory update, and selective assistant-context recall without storing full histories in the model context. Across LoCoMo-10 and LongMemEval-S, DimMem achieves **81.43%** and **78.20%** overall accuracy, respectively, outperforming existing lightweight memory systems while reducing LoCoMo per-query token cost by **24%**. We further show that dimensional memory extraction is learnable by compact models: after fine-tuning on the DimMem schema, a Qwen3-4B extractor surpasses LightMem with GPT-4.1-mini on both benchmarks and reaches performance comparable to, or better than, much larger extractors in key settings. These results suggest that explicit dimensional structuring is an effective and efficient foundation for long-term memory in LLM agents.

🔗 **Code for DimMem:** anonymous.4open.science/r/DimMem-3E84

1 Introduction

Large language model (LLM) agents are increasingly expected to act as persistent assistants: they must remember user preferences, past decisions, evolving facts, prior recommendations, and temporally grounded events across many sessions. This requirement has motivated a broad family of long-term memory systems that externalize interaction histories into retrievable stores [1, 2, 3]. Recent systems further improve efficiency by compressing dialogues into facts, summaries, linked memory units, or staged memory stores [4, 5, 6, 7]. However, results on long-term conversational benchmarks such as LoCoMo and LongMemEval show that agent memory is not merely a problem of storing more text or retrieving semantically similar passages [8, 9]. Queries often depend on *when* something happened, *why* a user changed a plan, *where* an event occurred, *which* person or object was involved, or whether the user is asking about a previous assistant response.

Existing memory systems therefore face a fidelity–efficiency dilemma. Raw dialogue or large text chunks preserve context, but they are expensive to store, retrieve, update, and place into the model context. Flat extracted facts are efficient, but they often discard the temporal, causal, intentional, and contextual information needed for precise recall. More structured designs, such as graph-based or hierarchical memory systems, can recover relational structure, but they introduce additional

*Corresponding authors. Fanyi Wang is the project leader.

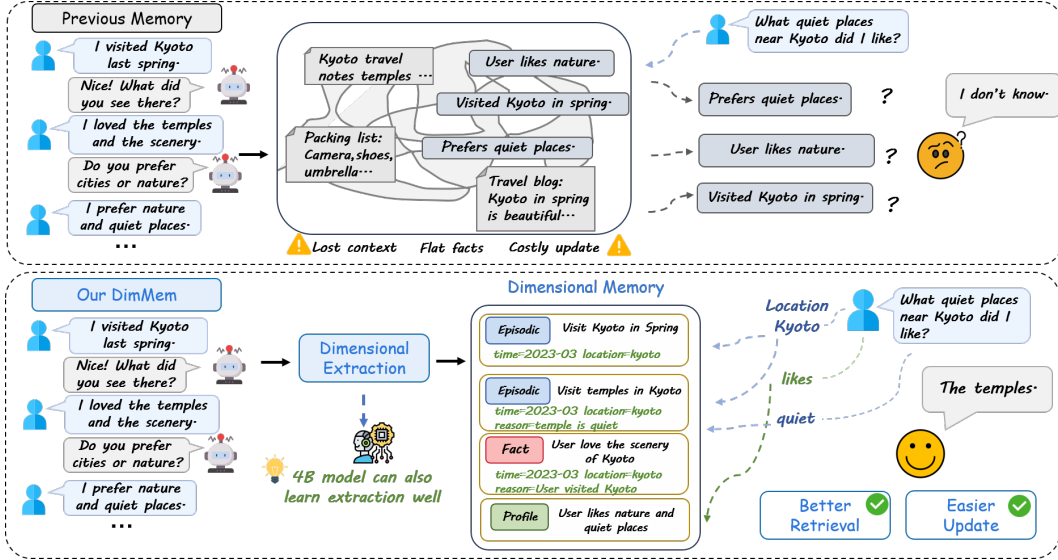


Figure 1: Conceptual motivation of DimMem. Prior memory systems often store conversations as raw, flat, or entangled memory forms, making retrieval and update depend on incomplete or costly context. DimMem instead exposes key dimensions within each memory unit, enabling more direct retrieval, easier update, and compact extractor learning.

construction and maintenance cost [10, 11, 12, 13]. A parallel line of work learns memory operations through parameterized policies, using supervised fine-tuning or reinforcement learning to decide when to create, read, update, or delete memories [14, 15, 16]. These approaches improve the control policy of memory management, but they still depend on the quality of the memory representation being controlled. If the stored unit does not expose time, reason, purpose, or memory type, downstream retrieval and update policies must infer these missing dimensions from an impoverished record.

Figure 1 summarizes our central observation: long-term agent memory should be structured before it is retrieved, updated, or learned by a memory controller. Inspired by cognitive distinctions among episodic, semantic, and personal semantic memory [17, 18, 19] and by event-oriented 5W1H information structure [20], we propose **DimMem**, a lightweight dimensional memory framework for LLM agents. DimMem represents each memory as an atomic, self-contained unit with an explicit memory type (fact, episodic, or profile) and decoupled dimensional fields including *time*, *location*, *reason*, *purpose*, and typed *keywords*. Rather than treating metadata as optional annotations, DimMem makes these fields the operational interface of the memory system.

This dimensional representation enables three system-level capabilities. First, DimMem performs dimension-aware retrieval by parsing a user query into the same schema as memory records and combining lexical matching, semantic matching, and dimension scoring. This tri-path retrieval prevents semantically related but temporally or causally mismatched memories from dominating the context. Second, DimMem supports dimension-aware update: memory type, keywords, temporal order, and causal/purpose fields provide cheap structural filters for identifying redundant or conflicting memories before expensive embedding or LLM-based consolidation. Third, DimMem introduces dynamic assistant-memory recall. Assistant replies are not blindly embedded into every memory record; instead, the query parser predicts whether assistant context is needed and retrieves the corresponding source replies only on demand.

DimMem also changes what is learned by a small model. Instead of training a model to choose memory CRUD actions, we train a compact extractor to produce high-quality dimensional memory records. This is important because memory extraction quality determines the substrate available to every later retrieval and update step. We fine-tune a 4B model on teacher-generated dimensional extraction examples and obtain a local structured extractor that learns the schema rather than merely imitating surface summaries. In our experiments, this fine-tuned 4B extractor reaches performance comparable to a much larger Qwen3-30B-A3B extractor, and in some settings even surpasses it, showing that dimensional memory extraction is a learnable capability rather than a privilege of proprietary or large-scale models.

Empirically, DimMem demonstrates that explicit dimensional structure is a strong inductive bias for agent memory. Across LoCoMo-10 and LongMemEval-S, DimMem consistently outperforms existing lightweight memory systems, with the largest gains appearing on dimension-dependent cases such as temporal reasoning, multi-session reasoning, knowledge updates, and assistant-dependent recall. Ablations further confirm that the dimensional schema, dimension retrieval route, and reason/purpose fields are not cosmetic metadata: they directly improve retrieval precision, memory update quality, and context efficiency.

Our contributions are summarized as follows:

- **Dimensional memory representation.** We introduce DimMem, a lightweight long-term memory framework that represents each memory as an atomic, typed, and dimensionally structured record rather than a flat passage, summary, or isolated fact.
- **Dimension-aware memory operations.** We design retrieval, update, and assistant-recall mechanisms that align parsed query intents with explicit memory dimensions, improving temporal, causal, multi-session, and assistant-dependent reasoning while reducing retrieval and update cost.
- **Learnable structured extraction.** We show that high-quality dimensional memory extraction can be learned by a compact SFT model, and that extraction quality is crucial to end-to-end memory performance: a 4B extractor can approach or surpass larger general-purpose extractors when trained on the DimMem schema.

2 Related Work

2.1 Memory Systems for LLM Agents

Long-term memory has long been studied through neural memory and retrieval-augmented models [21, 22, 23, 24, 25]. For LLM agents, however, memory must support not only factual recall but also personalization, temporal continuity, preference tracking, and knowledge updates across interactions. Existing agent memory systems externalize dialogue history into retrievable stores, ranging from archival memory and personalized memory banks to fact-based stores, OS-style memory management, associative memories, staged updates, and query-planned retrieval [1, 2, 3, 4, 26, 27, 5, 6, 7, 28]. Benchmarks such as LoCoMo and LongMemEval further show that long-term conversational memory requires temporal reasoning, multi-session tracking, knowledge updates, and assistant-side recall rather than simple semantic lookup [8, 9]. DimMem is positioned within this line of work, but improves the memory interface by making type, time, location, reason, purpose, and keywords explicit dimensions shared by extraction, retrieval, and update.

2.2 Parameterized Memory Learning

A growing line of work studies whether memory behavior should be learned rather than specified entirely by hand-written rules. Recent approaches use supervised fine-tuning and reinforcement learning to train memory managers that decide when to create, read, update, delete, or ignore memories [14, 15, 16, 29]. This trend reflects an important shift from static memory pipelines toward adaptive memory control, where the model learns operational decisions from data. Most of these methods parameterize the *control policy*: they focus on which memory action to execute and when. DimMem instead parameterizes a complementary component, the *memory construction function*: it uses SFT to teach a compact extractor to produce high-quality dimensional memory records, improving the structured substrate on which later retrieval, update, or learned memory policies operate.

2.3 Memory Storage Design and Management

Memory systems also differ in how they store and manage accumulated experience. Lightweight designs store short facts, self-contained memory units, or topic-aware segments to reduce context and storage cost [6, 7]. These representations are efficient, but their stored units often expose limited structure for temporal validity, causal background, task intent, or type-conditioned update behavior. Richer designs organize memories with graphs, hierarchies, associative links, event structures, note abstractions, or topology-structured banks [5, 10, 11, 12, 13]. These approaches can strengthen relational reasoning and consolidation, but they also introduce graph construction, maintenance,

and delayed-update overhead. DimMem takes a lighter-weight middle path: it avoids full graph or hierarchy construction, but embeds the structure needed for precise retrieval and update directly inside each atomic memory through explicit dimensions.

3 Methodology

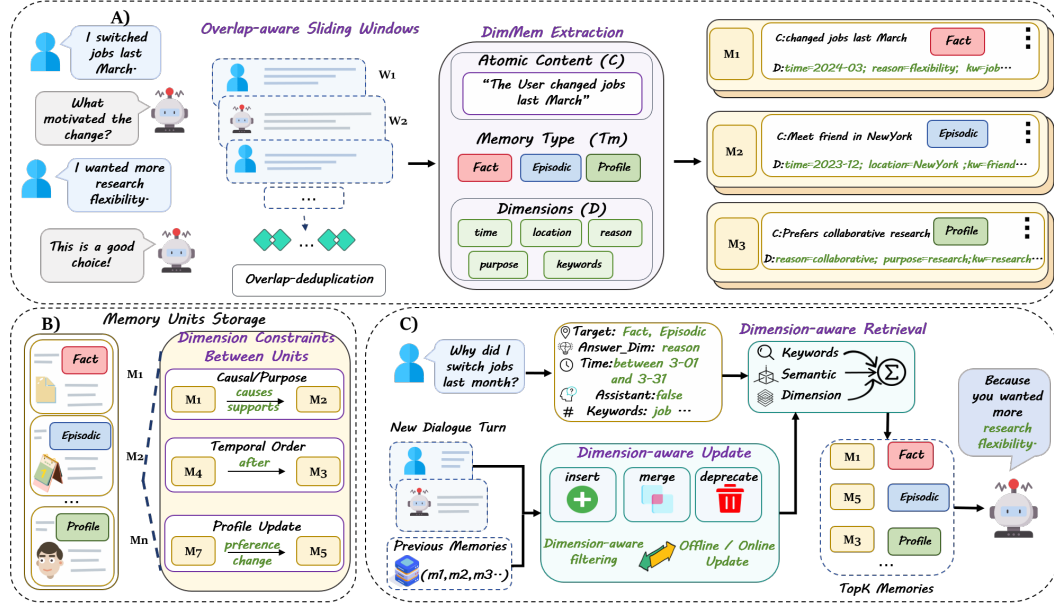


Figure 2: Overview of **DimMem**. (a) **Construction**: dialogue streams are segmented with overlap-aware windows and converted into atomic, typed, and dimensionally structured memory records. (b) **Storage**: explicit types and dimensions make cross-memory relations more discoverable, including temporal order, causal/purpose dependencies, and profile changes. (c) **Retrieval and update**: parsed queries use the same schema for tri-path retrieval, selective assistant-reply reconstruction, and dimension-aware insertion, merging, and deprecation.

DimMem follows a single principle: *operational memory dimensions are exposed at construction time*, as shown in Figure 2. Instead of storing dialogues as raw passages, flat facts, or opaque embeddings, DimMem converts interactions into atomic records with explicit dimensions. User queries are parsed into the same schema, enabling retrieval, assistant-context reconstruction, and memory update to share a unified interface. Thus, what to store, how to retrieve, and how to maintain memory are coupled through the same dimensional representation.

3.1 Dimensional Memory Records

Let a dialogue stream be $\mathcal{S} = \{(u_t, a_t, \tau_t)\}_{t=1}^T$, where u_t , a_t , and τ_t denote the user utterance, assistant utterance, and timestamp at turn t . DimMem maps $\mathcal{S}$ into a memory bank $\mathcal{B} = \{m_i\}_{i=1}^M$, where each memory is represented as

$$m_i = \langle id_i, C_i, T_i, \mathcal{D}_i, \rho_i \rangle, \quad (1)$$

where id_i is a unique identifier, C_i is a self-contained textual memory, T_i is the memory type, $\mathcal{D}_i$ contains explicit dimensions, and ρ_i points to the source window and turn. The pointer is not inserted into the QA context by default; it preserves access to grounded evidence, such as the original assistant reply, when a query explicitly requires it.

The type $T_i \in \{\text{fact}, \text{episodic}, \text{profile}\}$ distinguishes stable knowledge, event-specific experience, and long-term personal semantics [17, 19]. The dimensional field is

$$\mathcal{D}_i = \{D_{\text{time}}, D_{\text{loc}}, D_{\text{reason}}, D_{\text{purpose}}, D_{\text{kw}}\}. \quad (2)$$

Here D_{time} records when a memory is valid or occurred, D_{loc} records physical or contextual location, D_{reason} records causal background, D_{purpose} records communicative or task intent, and D_{kw} contains

entity and topic anchors. Empty fields are allowed when unsupported by the dialogue. This convention avoids hallucinated metadata while still exposing grounded structure whenever the conversation provides it.

3.2 Overlap-Aware Memory Extraction

Long dialogues are costly to process as a single prompt and may dilute memory-relevant signals. Inspired by prompt compression and lightweight memory-augmented generation [30, 6], DimMem first compresses dialogue content before memory extraction. This reduces token consumption while filtering redundant or task-irrelevant information, thereby improving the signal-to-noise ratio of extraction. The dialogue S is then partitioned into overlapping windows:

$$W_n = \{(u_t, a_t, \tau_t)\}_{t=s_n}^{e_n}, \quad s_{n+1} = e_n - O + 1, \quad (3)$$

where O denotes the overlap size. The overlap preserves local discourse context across adjacent windows, but should not introduce duplicate memories. We therefore impose an extraction invariant: tokens in the overlap may be used to resolve coreference, normalize relative time, and identify implicit event boundaries, whereas newly extracted memories must be grounded in the non-overlapping suffix:

$$\mathcal{M}_n = f_\theta(W_n; \text{context}(W_n \cap W_{n-1})), \quad \text{source}(m_i) \in W_n \setminus W_{n-1}. \quad (4)$$

For each extracted memory, the extractor outputs a normalized record containing `content`, `memory_type`, `time`, `location`, `reason`, `purpose`, and `keywords`. Each memory content C_i must be self-contained: pronouns are resolved, relative temporal expressions are normalized using τ_t , and unsupported dimensions are left empty. This yields a compact memory bank whose entries remain grounded through ρ_i while remaining queryable without replaying the raw dialogue.

Following the trade-off analysis in §C, we use window/overlap sizes of 15/3 for LongMemEval and 25/5 for LoCoMo to match their different turn densities.

3.3 Intent Recognition and Dimension-Aligned Retrieval

At inference time, DimMem maps a natural-language question into an operational memory intent before retrieval. This step identifies the requested answer dimension, the relevant memory type, the entity anchors, the dimensional constraints, and whether the user is asking about previous assistant behavior. Formally, given a query q , DimMem infers

$$\hat{q} = \langle A_q, T_q, \tilde{D}_q, E_q, C_q, I_{\text{ast}} \rangle, \quad (5)$$

where A_q denotes the answer dimension to recover, T_q is the target memory type when identifiable, $\tilde{D}_q$ contains constraints over the same dimensions used by memory records, E_q contains grounded entity anchors, C_q is a semantic query anchor, and $I_{\text{ast}} \in \{0, 1\}$ marks assistant-dependent intent. This representation separates *what to answer* from *which memories should be retrieved*. For example, a temporal constraint such as “after March” restricts retrieval, whereas a temporal answer request such as “when did this happen” sets $A_q = D_{\text{time}}$ without filtering out memories that contain the answer.

Tri-path retrieval. DimMem uses three complementary retrieval views. Lexical matching is reliable for exact entities and rare phrases, dense retrieval recovers semantic paraphrases, and dimension matching enforces explicit constraints over type, time, location, reason, or purpose. Prior retrieval-augmented systems commonly combine multiple recall signals to compensate for the blind spots of individual retrievers [31, 32, 33, 34]. DimMem adopts this multi-route strategy, but makes the third route a *dimension route* over the fields created by the extractor:

$$\mathcal{R}(q) = \text{Dedup}(\mathcal{R}_{\text{bm25}}(q) \cup \mathcal{R}_{\text{dim}}(q) \cup \mathcal{R}_{\text{dense}}(q)). \quad (6)$$

The lexical route $\mathcal{R}_{\text{bm25}}$ matches query anchors against memory content and textual dimensions. The dense route $\mathcal{R}_{\text{dense}}$ embeds query and memory text to recover paraphrases. The dimension route $\mathcal{R}_{\text{dim}}$ scores explicit agreement between query constraints and memory dimensions:

$$s_{\text{dim}}(q, m_i) = \sum_{r \in \mathcal{A}(q)} \bar{\lambda}_r s_r(q, m_i), \quad \bar{\lambda}_r = \frac{\lambda_r}{\sum_{r' \in \mathcal{A}(q)} \lambda_{r'}}. \quad (7)$$

$\mathcal{A}(q)$ is the set of active constraints in the query, and weights are normalized over active dimensions so that absent fields do not dilute the score. The final list preserves evidence from all three routes while removing duplicate memory contents. This design makes dimensional structure operational: it does not replace lexical or semantic retrieval, but constrains them when the query specifies when, where, why, or what type of memory is needed.

Assistant-dependent intent. The intent variable I_{ast} controls whether the retrieval context should include previous assistant replies. Some questions ask what the assistant recommended, explained, or decided, while most questions can be answered from user-side memories alone. Storing all assistant replies in every memory record would inflate the memory bank and QA context, so DimMem treats assistant content as recoverable evidence. If $I_{\text{ast}} = 1$, each retrieved memory m_i is mapped back through ρ_i , and the corresponding assistant reply is attached:

$$\mathcal{C}_{\text{aug}} = \{m_i \oplus \Phi_{\text{ast}}(\rho_i) \mid m_i \in \mathcal{R}(q), I_{\text{ast}} = 1\}, \quad (8)$$

where Φ_{ast} retrieves the original assistant utterance. If $I_{\text{ast}} = 0$, only structured memory records are used. Dynamic assistant recall is therefore part of intent handling: assistant context remains available when the intent requires it, but does not become a constant cost for all queries.

3.4 Dimension-Aware Memory Update

As the memory bank grows, new records must be inserted, merged, used to supersede outdated records, or kept separate. Purely embedding-based updates may miss conflicts with different wording or over-merge semantically similar but temporally distinct memories. DimMem therefore uses dimensions as a cheap candidate-generation layer: records are first grouped by memory type and filtered by typed keyword overlap, then verified by embedding similarity before LLM consolidation. Thus, expensive LLM decisions are invoked only for structurally plausible candidate pairs.

For each candidate pair $(m_{\text{new}}, m_{\text{old}})$, an LLM-based consolidator selects one action:

$$a \in \{\text{MERGE}, \text{SUPERSEDE}, \text{KEEPBOTH}\}. \quad (9)$$

The decision prompt exposes both content and dimensions, with type-conditioned semantics: for fact records, newer contradictory facts may SUPERSEDE older ones, while complementary facts may MERGE; for episodic records, repeated descriptions of the same event may MERGE, whereas distinct events, especially those occurring at different times, should KEEP BOTH; for profile records, overlapping preferences, habits, or style descriptions may MERGE, while unrelated profile aspects remain separate. This update policy keeps the memory bank compact without erasing distinct episodes, and makes consolidation auditable by grounding decisions in explicit dimensions rather than hidden embedding similarity alone.

3.5 Learning the Structured Extractor

DimMem’s schema is useful only if it can be populated reliably. We therefore train a compact extractor to map dialogue windows into structured memory records, rather than relying on a large proprietary model at inference time. To obtain supervision without using benchmark test annotations, we construct synthetic training data with GPT-5.4 as the teacher. Specifically, we generate 5K LoCoMo-style examples and 5K LongMemEval-style examples. The LongMemEval-style examples are generated from the UltraChat training split [35], by sampling dialogue windows and annotating them according to the DimMem schema. We filter malformed, empty, or schema-inconsistent outputs before fine-tuning.

We fine-tune Qwen3-4B [36] with LoRA [37]. The system prompt defines the memory taxonomy, field meanings, time normalization, coreference resolution, and the rule that unsupported dimensions remain empty. Each training sample follows the chat-completion format:

$$x_i = [s_{\text{schema}}, W_i, \mathcal{M}_i^{\text{teacher}}], \quad (10)$$

where W_i is the dialogue window and $\mathcal{M}_i^{\text{teacher}}$ is the teacher-generated memory JSON. After fine-tuning, the LoRA adapters are merged into the base model and served with vLLM [38]. The trained extractor is then used as a drop-in component in the same DimMem pipeline, allowing us to test whether dimensional memory construction can be learned by a compact model rather than only elicited through large-model prompting.

Table 1: QA accuracy on the LoCoMo-10 benchmark. Best and second-best results within each model block are marked in **bold** and underline, respectively. For Tokens/Query, lower is better.

Method	MultiHop	Temporal	OpenDomain	SingleHop	Overall	Tokens/Query
 GPT-4.1-mini						
FullText	59.22	68.22	50.00	72.41	67.72	-
NaiveRAG	61.35	37.07	41.67	74.67	62.34	10,234
MemoryOS	62.77	39.25	43.75	76.69	64.28	8,798
MemOS	70.57	40.19	48.96	82.28	69.29	7,457
A-Mem	63.12	40.81	50.00	78.00	65.78	9,743
LightMem	62.77	64.17	54.17	81.57	72.79	<u>5,063</u>
SimpleMem	68.44	63.86	59.38	86.44	76.75	<u>6,527</u>
DimMem (Ours)	81.91	81.62	55.21	82.50	<u>80.51</u>	3,859
DA-Update	<u>80.85</u>	<u>81.00</u>	<u>55.21</u>	<u>84.78</u>	81.43	-
 Qwen3-30B-A3B-Instruct-2507						
FullText	56.74	46.73	<u>57.29</u>	87.75	71.62	-
NaiveRAG	57.45	40.19	55.21	77.29	64.55	11,243
MemoryOS	56.03	42.99	47.92	69.32	60.06	8,537
MemOS	59.93	39.56	53.13	77.65	64.94	7,537
A-Mem	53.19	41.12	55.21	66.11	57.86	10,347
LightMem	61.70	60.75	51.04	79.31	70.45	<u>6,721</u>
SimpleMem	64.54	62.31	56.25	80.26	72.14	<u>6,993</u>
DimMem (Ours)	<u>75.53</u>	69.16	58.33	81.09	<u>76.17</u>	5,846
DimMem-4B	76.95	81.31	57.29	<u>82.88</u>	79.87	-
DimMem-4B†	72.70	<u>75.70</u>	46.88	76.10	73.57	-

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate DimMem on two long-term conversational memory benchmarks: **LoCoMo-10** [8], which emphasizes temporal, multi-hop, open-domain, and single-hop reasoning over very long dialogues, and **LongMemEval-S** [9], which stresses knowledge updates, multi-session tracking, temporal reasoning, user-preference recall, and assistant-side recall.

Implementation Details. We distinguish the *memory extractor* from the *QA model* to test whether DimMem’s schema can be learned by a compact extractor without changing downstream QA. Unless otherwise specified, dense retrieval uses MiniLM sentence embeddings [39, 32], and evaluation uses GPT-4.1-mini as the LLM judge [40]. We compare against raw-context, retrieval-based, and memory-system baselines, including MemoryOS [26], MemOS [27], A-Mem [5], LightMem [6], and SimpleMem [7]. **DimMem-4B** uses a fine-tuned Qwen3-4B extractor with GPT-4.1-mini for QA, while **DimMem-4B†** uses fine-tuned Qwen3-4B for extraction and Qwen3-4B for QA, representing a fully compact local setting. All experiments are conducted on a single NVIDIA A800 GPU (80GB).

4.2 Results and Analysis

Tables 1 and 2 present the main results. We focus on three questions: whether dimensional memories improve long-term reasoning, whether the update and assistant-recall mechanisms address benchmark-specific failure modes, and whether a compact model can learn to construct useful dimensional memories. KU=Knowledge-Update, MS=Multi-Session, SSA=Single-Session-Assistant, SSP=Single-Session-Preference, SSU=Single-Session-User, TR=Temporal-Reasoning.

Dimensional memories improve long-term reasoning. DimMem consistently outperforms memory baselines across both benchmarks. The gains are most visible on LoCoMo-10, where questions often require combining entities, events, and temporal context across long conversations. The result supports our central hypothesis: storing compact memories is not enough; the stored unit must expose the dimensions needed by later retrieval and reasoning.

Table 2: QA accuracy on the LongMemEval-S benchmark. Best and second-best results within each model block are marked in **bold** and underline, respectively.

Method	TR	MS	KU	SSU	SSA	SSP	Overall
🌀 GPT-4.1-mini							
FullText	31.58	46.62	43.59	80.00	41.07	56.67	46.80
NaiveRAG	57.14	54.14	74.36	77.14	60.71	53.33	62.00
MemoryOS	50.38	48.12	71.79	85.71	51.79	66.67	59.20
MemOS	63.91	54.14	70.51	90.00	64.29	50.00	65.20
A-Mem	58.65	52.63	69.23	88.57	57.14	46.67	62.00
SimpleMem	<u>81.95</u>	57.89	74.36	82.86	71.43	<u>76.67</u>	73.00
LightMem	84.21	51.13	<u>88.46</u>	90.00	23.21	76.67	69.60
DimMem (Ours)	74.44	<u>64.66</u>	<u>84.62</u>	<u>98.57</u>	76.79	63.33	<u>76.40</u>
DA-Update	77.44	69.17	88.46	98.57	<u>73.21</u>	56.67	78.20
🌀 Qwen3-30B-A3B-Instruct-2507							
FullText	63.91	42.11	65.38	71.43	32.14	46.67	54.80
NaiveRAG	65.41	48.87	71.79	81.43	30.36	53.33	59.60
MemoryOS	57.14	40.60	64.10	61.43	28.57	50.00	50.80
MemOS	72.93	51.13	73.08	80.00	26.79	53.33	61.80
A-Mem	66.92	44.36	71.79	74.29	35.71	63.33	59.00
SimpleMem	74.44	52.63	75.64	<u>87.14</u>	<u>64.29</u>	<u>66.67</u>	<u>69.00</u>
LightMem	<u>75.94</u>	57.89	89.74	85.71	21.43	73.33	68.40
DimMem (Ours)	76.69	<u>54.89</u>	<u>79.49</u>	94.29	73.21	56.67	72.20
DimMem-4B	73.68	<u>67.42</u>	83.33	90.00	46.43	60.00	71.80
DimMem-4B†	76.69	55.64	83.33	84.29	42.86	56.67	68.20

Dimension-aware operations target the hard cases. DA-Update improves the cases where memories evolve or conflict over time, especially multi-session and knowledge-update questions. Dynamic assistant recall addresses a different failure mode: some queries ask about what the assistant previously recommended or explained. Instead of storing all assistant responses in every memory, DimMem retrieves assistant context only when the query requires it, which improves assistant-dependent recall without increasing the default memory context.

Fine-tuning makes compact extraction viable. DimMem-4B shows that the dimensional extraction skill can be learned by a small model. With the fine-tuned Qwen3-4B extractor and GPT-4.1-mini QA, DimMem-4B reaches performance comparable to, and in LoCoMo-10 even higher than, DimMem using the larger Qwen3-30B-A3B extractor. DimMem-4B† is lower because both extraction and QA are handled by Qwen3-4B, but it remains a useful fully compact configuration. This separation shows that much of the gain comes from learning to construct high-quality dimensional memories, while the remaining gap is largely due to the QA model.

The main results show that DimMem improves long-term conversational memory. This section isolates the design choices behind these gains: whether dimensional records improve the stored representation, whether the dimension route adds recall beyond lexical and dense retrieval, and whether dimensions reduce the cost of assistant recall and memory update. We further provide case studies in §A. Additional segmentation and update analysis is provided in §4.5 and §C.

4.3 Ablation Study

We evaluate three controlled variants using GPT-4.1-mini as the backbone model. **Content-Only Memory** collapses each record into flat text, isolating the value of dimensional representation. **Lexical+Dense Retrieval** keeps dimensional records but removes dimension-based scoring, isolating the value of query-memory dimension alignment. **No Causal/Intent Dimensions** removes reason and purpose while preserving type, time, location, and keywords, isolating the contribution of causal and intentional fields.

Table 3: Ablation results on LongMemEval-S (%). Absolute changes are relative to DimMem (Full).

Config	KU	MS	SSA	SSP	SSU	TR	Overall
DimMem (Full)	88.46	69.17	73.21	56.67	98.57	77.44	78.20
Content-Only Memory	85.90 _{↓2.56%}	61.65 _{↓7.52%}	17.86 _{↓55.35%}	63.33 _{↑6.66%}	98.57 _{→0.00%}	75.94 _{↓1.50%}	69.60 _{↓8.60%}
Lexical+Dense Retrieval	87.18 _{↓1.28%}	62.41 _{↓6.76%}	76.79 _{↑3.58%}	60.00 _{↑3.33%}	97.14 _{↓1.43%}	72.18 _{↓5.26%}	75.20 _{↓3.00%}
No Causal/Intent Dims.	88.46 _{→0.00%}	69.92 _{↑0.75%}	76.79 _{↑3.58%}	53.33 _{↓3.34%}	97.14 _{↓1.43%}	72.18 _{↓5.26%}	77.00 _{↓1.20%}

Table 4: Ablation results on LoCoMo-10 (%). Absolute changes are relative to DimMem (Full).

Config	Multi-hop	Temporal	Open-domain	Single-hop	Overall
DimMem (Full)	80.85	81.00	55.21	84.78	81.43
Content-Only Memory	62.77 _{↓18.08%}	64.17 _{↓16.83%}	52.08 _{↓3.13%}	78.60 _{↓6.18%}	71.04 _{↓10.39%}
Lexical+Dense Retrieval	81.02 _{↑0.17%}	77.00 _{↓4.00%}	53.26 _{↓1.95%}	83.47 _{↓1.31%}	79.87 _{↓1.56%}
No Causal/Intent Dims.	80.85 _{→0.00%}	79.75 _{↓1.25%}	55.21 _{→0.00%}	83.69 _{↓1.09%}	80.58 _{↓0.85%}

Tables 3 and 4 show that the representation is the dominant factor: Content-Only Memory causes the largest overall drop on both benchmarks, especially on LoCoMo-10 multi-hop and temporal questions and on LongMemEval-S assistant-dependent questions. Lexical+Dense Retrieval produces smaller but consistent losses, showing that BM25 and dense retrieval recover many candidates but cannot directly enforce time, type, and location constraints. Removing causal and intentional fields has the smallest overall effect, but still hurts temporal reasoning, suggesting that reason and purpose are most useful when lexically similar memories differ in cause or intent.

4.4 Dynamic Assistant Recall: Selective Context Reconstruction

Some questions ask about previous assistant replies rather than user-stated facts. Storing all assistant replies in every memory record would inflate all queries, so DimMem predicts whether assistant context is needed and reconstructs it through source pointers only on demand.

Figure 3 shows that selective recall captures most of the benefit of assistant context at a much lower cost. It improves overall accuracy by 6.0 points, with the largest gain on Single-Session-Assistant questions (+48.2), while triggering assistant-reply lookup for only 12.2% of queries. This reduces token consumption to 34.4% of full recall (2.06M vs. 5.99M tokens). We further analyze the detection quality of the recall trigger in §4.6. The detector achieves 90.2% precision, 98.2% recall, and 94.0% F1, missing only one assistant-dependent query, which explains why selective recall remains robust while avoiding unnecessary assistant-context reconstruction.

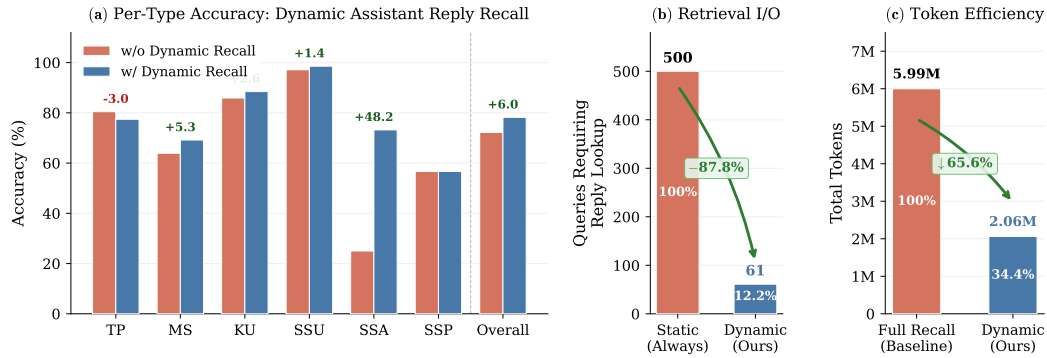


Figure 3: Dynamic assistant recall on LongMemEval-S. The mechanism improves assistant-dependent questions while triggering reply lookup for only a small fraction of all queries.

4.5 Update Efficiency: Dimensions Reduce Candidate Search

Memory update becomes expensive when every new record is compared against the full bank. We compare DimMem with a LightMem-style embedding baseline. DimMem first filters candidates by

memory_type and keyword overlap, then applies embedding verification and LLM consolidation only to the reduced set.

Table 5: Memory update efficiency. DimMem uses dimensional filters before embedding and LLM consolidation.

Dataset	Method	Embed. Count	Embed. Saved	Merges	Compression
LongMemEval	LightMem	110,601 (100%)	—	5,897	5.33%
	DimMem	27,107 (24.5%)	75.5%	7,555	6.83%
LoCoMo	LightMem	2,475 (100%)	—	241	9.74%
	DimMem	1,011 (40.8%)	59.2%	247	9.98%

Table 5 shows that dimensional filtering reduces embedding computation by 75.5% on LongMemEval and 59.2% on LoCoMo, while finding slightly more merges than the baseline. The extra merges come from pairs that share memory type and typed keywords but fall below the baseline’s high embedding threshold. Thus, dimensions act as cheap filters that improve both efficiency and candidate quality before expensive LLM consolidation.

4.6 Detection Quality of Dynamic Assistant Recall

To further examine whether dynamic assistant recall reliably identifies assistant-dependent questions, we report the detection confusion matrix in Table 6. The ground truth is whether the query belongs to the assistant-dependent Single-Session-Assistant (SSA) type.

Table 6: Detection quality for dynamic assistant recall on LongMemEval-S. Ground truth is whether the query belongs to the assistant-dependent SSA type.

	Actual Need	Actual No Need	Total
Predicted Need	55	6	61
Predicted No Need	1	438	439
Total	56	444	500

Precision = 90.2%, Recall = 98.2%, F1 = 94.0%.

Table 6 shows that the recall trigger is highly reliable. It misses only one assistant-dependent query and introduces only six false positives. The false positives mainly increase a small amount of irrelevant context, whereas the low false-negative rate ensures that nearly all assistant-dependent questions receive the required source-level reply reconstruction. This supports the design choice of storing source pointers rather than duplicating assistant replies inside every memory record.

5 Conclusion and Limitations

We presented **DimMem**, a lightweight dimensional memory framework for long-term LLM agents. DimMem stores memories as atomic, typed records with explicit dimensions, enabling extraction, query parsing, retrieval, assistant recall, and update to share one schema. Experiments on LoCoMo-10 and LongMemEval-S show improved accuracy and efficiency, while compact-extractor results suggest that dimensional memory construction can be learned by smaller models. **Limitations.** DimMem is evaluated only on textual conversational memory, and should be further tested in deployed, multimodal, and privacy-sensitive agents. It also depends on accurate extraction and intent parsing, and its schema may require domain-specific extensions. Update consolidation still relies on LLM judgments.

References

- [1] Joon Sung Park, Joseph C. O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, UIST 2023*, pages 2:1–2:22. ACM, 2023.
- [2] Wanjun Zhong, Lianhong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024*, pages 19724–19731. AAAI Press, 2024.
- [3] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *CoRR*, abs/2310.08560, 2023.
- [4] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *CoRR*, abs/2504.19413, 2025.
- [5] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *CoRR*, abs/2502.12110, 2025.
- [6] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. LightMem: Lightweight and efficient memory-augmented generation. *CoRR*, abs/2510.18866, 2025.
- [7] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. SimpleMem: Efficient lifelong memory for LLM agents. *CoRR*, abs/2601.02553, 2026.
- [8] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024*, pages 13851–13870. Association for Computational Linguistics, 2024.
- [9] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *CoRR*, abs/2410.10813, 2024.
- [10] Zhaofen Wu, Hanrong Zhang, Fulin Lin, Wujiang Xu, Xinran Xu, Yankai Chen, Henry Peng Zou, Shaowen Chen, Weizhi Zhang, Xue Liu, Philip S. Yu, and Hongwei Wang. GAM: Hierarchical graph-based agentic memory for LLM agents. *CoRR*, abs/2604.12285, 2026.
- [11] Ningning Zhang, Xingxing Yang, Zhizhong Tan, Weiping Deng, and Wenyong Wang. HiMem: Hierarchical long-term memory for LLM long-horizon agents. *CoRR*, abs/2601.06377, 2026.
- [12] Buqiang Xu, Yijun Chen, Jizhan Fang, Ruobin Zhong, Yunzhi Yao, Yuqi Zhu, Lun Du, and Shumin Deng. StructMem: Structured memory for long-horizon behavior in LLMs. *CoRR*, abs/2604.21748, 2026.
- [13] Can Lv, Heng Chang, Yuchen Guo, Shengyu Tao, and Shiji Zhou. All-Mem: Agentic lifelong memory via dynamic topology evolution. *CoRR*, abs/2603.19595, 2026.
- [14] Yupeng Huo, Yaxi Lu, Zhong Zhang, Haotian Chen, and Yankai Lin. AtomMem: Learnable dynamic agentic memory with atomic memory operation. *CoRR*, abs/2601.08323, 2026.
- [15] Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *CoRR*, abs/2601.01885, 2026.
- [16] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *CoRR*, abs/2508.19828, 2025.

- [17] Endel Tulving. Memory and consciousness. *Canadian Psychology / Psychologie canadienne*, 26(1):1–12, 1985.
- [18] Endel Tulving. Episodic memory: From mind to brain. *Annual Review of Psychology*, 53(1):1–25, 2002.
- [19] Louis Renoult, Patrick S. R. Davidson, Daniela J. Palombo, Morris Moscovitch, and Brian Levine. Personal semantics: At the crossroads of semantic and episodic memory. *Trends in Cognitive Sciences*, 16(11):550–558, 2012.
- [20] Felix Hamborg, Corinna Breiteringer, and Bela Gipp. Giveme5W1H: A universal system for extracting main events from news articles. *CoRR*, abs/1909.02766, 2019.
- [21] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *3rd International Conference on Learning Representations, ICLR 2015*, 2015.
- [22] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, NeurIPS 2015*, pages 2440–2448, 2015.
- [23] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020*, volume 119 of *Proceedings of Machine Learning Research*, pages 3929–3938. PMLR, 2020.
- [24] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *8th International Conference on Learning Representations, ICLR 2020*. OpenReview.net, 2020.
- [25] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning, ICML 2022*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240. PMLR, 2022.
- [26] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025*, pages 25961–25970. Association for Computational Linguistics, 2025.
- [27] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, Qingchen Yu, Jihao Zhao, Yezhaohui Wang, Peng Liu, Zehao Lin, Pengyuan Wang, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhen Tao, Huayi Lai, Hao Wu, Bo Tang, Zhengren Wang, Zhaoxin Fan, Ningyu Zhang, Linfeng Zhang, Junchi Yan, Mingchuan Yang, Tong Xu, Wei Xu, Huajun Chen, Haofen Wang, Hongkang Yang, Wentao Zhang, Zhi-Qin John Xu, Siheng Chen, and Feiyu Xiong. MemOS: A memory OS for AI system. *CoRR*, abs/2507.03724, 2025.
- [28] Zhanghao Hu, Qinglin Zhu, Di Liang, Hanqi Yan, Yulan He, and Lin Gui. Beyond RAG for agent memory: Retrieval by decoupling and aggregation. *CoRR*, abs/2602.02007, 2026.
- [29] Kehao Zhang, Shangdong Gui, Sheng Yang, Wei Chen, and Yang Feng. Learning to remember: End-to-end training of memory agents for long-context reasoning. *CoRR*, abs/2602.18493, 2026.
- [30] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMingua: Compressing prompts for accelerated inference of large language models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, Singapore, December 2023. Association for Computational Linguistics.

- [31] Stephen E. Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [32] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019*, pages 3982–3992. Association for Computational Linguistics, 2019.
- [33] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick S. H. Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020*, pages 6769–6781. Association for Computational Linguistics, 2020.
- [34] Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020*, 2020.
- [35] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3029–3051, Singapore, December 2023. Association for Computational Linguistics.
- [36] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025.
- [37] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations, ICLR 2022*. OpenReview.net, 2022.
- [38] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023*, pages 611–626. ACM, 2023.
- [39] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020*, 2020.
- [40] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*, 2023.

APPENDIX

A Case Study

We walk through a representative retrieval example end-to-end to illustrate three operational stages of DimMem: (1) query parsing into explicit dimensions, (2) dimension-aware scoring, and (3) structured memory records presented to the reader model. This is followed by a second, shorter case showing how dimension-aware update resolves conflicting memory records.

A.1 Case 1: End-to-End Dimension-Aware Retrieval

Query. “How many days did I spend on camping trips in the United States this year?”

Question date. 2023/04/29 **Gold answer.** 8 days.

A.1.1 Step 1: Dimensional Query Parsing

The query analyzer maps the natural-language question into a structured retrieval specification. Below is the actual parsed output:

Table 7: Parsed query dimensions produced by the query analyzer.

Field	Parsed Value
query_anchor	“The number of days the user spent on camping trips in the United States in 2023”
target_memory_type	[episodic]
keywords	[“camping trips”, “United States”]
time	between 2023-01-01 and 2023-04-29
location	United States
answer_dim	content
need_assistant_context	false

The query analyzer makes several important decisions:

- It converts the relative phrase “this year” to the constraint between 2023-01-01 and 2023-04-29, anchored to the question date.
- It identifies the memory type episodic (trip events) rather than fact or profile.
- It extracts “United States” as both a keyword and location constraint.
- It determines that the answer is in memory content (number of days) rather than in time or location fields.

A.1.2 Step 2: Dimension-Aware Scoring

The dimension route scores each memory record against the parsed constraints. Table 8 shows the per-component breakdown for two candidate records. Only dimensions present in the query are activated.

Table 8: Dimension scoring breakdown for two candidate memory records.

Component	Weight $\bar{\lambda}_r$	Record A	Record B
memory_type (target: episodic)	0.158	1.0 ✓	1.0 ✓
time_constraint (between 2023-01 and 2023-04-29)	0.316	1.0 ✓	0.0 ✗
location (United States)	0.211	0.0	1.0 ✓
keyword_phrase (“camping trips”, “United States”)	0.158	0.0	0.0
keyword_token_overlap	0.158	0.50	0.25
Final s_{dim}	—	0.553	0.408

Record A (rank 1 in dimension route): a 3-day camping trip to Big Sur in early April 2023. It scores high because its type is episodic and its source timestamp (2023-04-29) falls within the query’s time

Table 9: Top-3 records from the dimension route for the camping query. Each record carries the same dimensional schema used at write time, enabling structured matching at query time.

#	Content	Dimensions
1	The user recently returned from a 3-day solo camping trip to Big Sur in early April and found their hiking boots adequate but thinks they need waterproof boots for multi-day backpacking trips.	type: episodic time: early April 2023 location: Big Sur reason: need for better boots after trip purpose: find waterproof boots for multi-day backpacking keywords: [Big Sur, solo camping, hiking boots, waterproof boots, multi-day backpacking]
2	The user recently completed an amazing 5-day camping trip to Yellowstone National Park in March 2023 and is still enthusiastic about the experience.	type: episodic time: 2023-03 location: Yellowstone National Park reason: — purpose: — keywords: [Yellowstone National Park, 5-day camping trip, outdoor experience]
3	The user was interested in visiting nearby national parks and requested recommendations for day trips near Moab, Utah.	type: episodic time: 2023-04-29 location: Moab, Utah reason: — purpose: — keywords: [national parks, day trips, Moab Utah, short trips]

window. The location field contains “Big Sur” but not the string “United States”, so the location sub-score is 0—yet the time and type signals are strong enough to rank it above records without temporal grounding.

Record B: planning a trip to Rocky Mountains in Colorado (2023-04-29). It matches location (“Colorado” is not the exact string “United States”) but its source timestamp falls on the boundary. This illustrates how multi-component scoring selects records with the best aggregate dimensional fit rather than relying on any single dimension.

A.1.3 Step 3: Retrieved Structured Memories

Table 9 shows the top-3 records from the dimension route, demonstrating how each record exposes its full dimensional schema.

Contrast with BM25-only. Without the dimension route, BM25 ranks a Passover-related record at position 4 (matching “number” and common function words) and pushes the Yellowstone camping record lower. The dimension route’s time constraint and type filter jointly ensure that both camping trip records appear in the top-5, enabling the reader model to sum the durations correctly ($3 + 5 = 8$ days).

A.2 Case 2: Dimension-Aware Memory Update

Scenario. During conversation, the user first expressed a general goal—reaching the Gold level on their Starbucks Rewards app—and later corrected the requirement as **125 stars** rather than 400. DimMem’s update pipeline detects this conflict before retrieval.

A.2.1 Multi-Level Candidate Detection

Before and after update.

- **Old:** “The user is trying to reach the Gold level on their Starbucks Rewards app and wants to know how many stars are needed.”

Table 10: Dimension-aware update: the multi-level filter identifies a candidate pair that purely embedding-based deduplication would miss.

Stage	Operation
Level 1: Type grouping	Both records are <code>memory_type=fact</code> $\rightarrow$ same-type candidate pair formed.
Level 2: Keyword overlap	Keywords intersection: {"Starbucks", "Gold", "stars"}; Jaccard $> 0.3 \rightarrow$ passed.
Level 3: Embedding	Cosine similarity = 0.797, above DimMem threshold 0.7 $\rightarrow$ passed. (Note: <i>LightMem</i> uses threshold 0.85; this pair would be missed.)
Level 4: LLM decision	Same entity (Starbucks Gold level); values conflict (vague $\rightarrow$ 125). Action: SUPERSEDE . Old memory archived; corrected memory retained.

- **New (retained):** "The user corrected that **125 stars** are needed to reach Gold level on the Starbucks Rewards app, not 400."

A.2.2 Why Dimensions Matter for Update

The multi-level pipeline uses three key observations:

1. **Type grouping** eliminates the vast majority of pairwise comparisons: only records of the same memory type can conflict.
2. **Keyword overlap** acts as a cheap entity-level filter: "Starbucks" + "Gold" + "stars" strongly suggests these two records are about the same entity.
3. **A lower embedding threshold** (0.7 vs. the typical 0.85) is safe because it is applied *after* the keyword filter has already ensured topical relatedness.

Without this multi-level design, either the pair goes undetected (threshold too high) or all memory pairs must be checked by the LLM consolidator (no keyword pre-filter). DimMem’s cascaded approach achieves high recall on true conflicts while keeping LLM calls manageable.

B Hyperparameters

C Additional Segmentation Analysis

DimMem extracts memories from overlapping dialogue windows. This design choice affects both extraction quality and extraction cost: small windows preserve local topic coherence but may not contain enough context for resolving pronouns, relative time expressions, or implicit event boundaries; large windows provide more context but increasingly mix unrelated events; excessive overlap repeats work and may create duplicate extraction opportunities. To study this trade-off, we evaluate 23 configurations with $W \in \{10, 15, 20, 25, 30, 35, 40, 50\}$ and overlap ratios from 0% to 60% on LongMemEval and LoCoMo.

Figure 4 examines the window-size side of the trade-off. Coherence decreases quickly as the window grows from $W = 10$ to $W = 20$, then becomes relatively flatter after $W = 25$. This indicates that increasing the window size provides additional context but also introduces more topic mixing. We therefore choose dataset-specific window sizes according to the conversation structure. For LongMemEval, we use $W = 15$ because its turns often contain longer messages and assistant replies, so a smaller number of turns already provides sufficient contextual evidence while avoiding excessive cross-topic mixing. For LoCoMo, where conversations are typically composed of shorter exchanges, we use $W = 25$ to preserve enough local continuity. DimMem favors smaller and more coherent windows because its target unit is an atomic dimensional memory rather than a topic-level summary.

Figure 5 studies the complementary overlap choice. In both datasets, a moderate overlap improves boundary continuity while keeping extraction overhead limited. Specifically, we use $W = 15, O = 3$

Table 11: Pipeline and model hyperparameters used in LongMemEval and LoCoMo.

Hyperparameter	LongMemEval	LoCoMo
Segmentation		
Window size	15	25
Overlap	3	5
Compression (LLMLingua-2)		
Compression rate	0.8	0.8
Compression model	bert-base-multilingual-cased -meetingbank	
Memory Extraction		
LLM	GPT-4.1-mini	Qwen3-30B-A3B-Instruct-2507
Temperature	0.0	0.0
Max output tokens	16,384	16,384
Query Analysis		
Temperature	0.0	0.0
Max output tokens	4,096	4,096
Retrieval		
Top- k per route	20	15
Embedding model	all-MiniLM-L6-v2 (384-dim)	
QA & Judge		
LLM	gpt-4.1-mini	
Temperature	0.0	0.0
Max output tokens (Judge)	512	512
LoRA Fine-tuning (Qwen3-4B)		
Precision	bfloat16	
Epochs	2	
Per-device batch size	1	
Gradient accumulation steps	2	
Effective batch size	2	
Learning rate	5×10^{-5}	
Max sequence length	12,288	
LoRA rank (r)	16	
LoRA alpha (α)	32	

for LongMemEval and $W = 25$, $O = 5$ for LoCoMo; both correspond to a 20% overlap ratio. This choice captures most of the boundary-continuity benefit without incurring the repeated extraction cost caused by larger overlaps. The LongMemEval setting is smaller in terms of message count because its messages and assistant replies are often longer, making each window more token-dense. The LoCoMo setting uses a larger window to compensate for shorter turns and preserve enough local context. This supports the overlap-aware extraction rule in Section 3.2: overlap should be used to ground coreference and time normalization, while new memories should be extracted only from the non-overlapping portion.

D LLM Usage Statement

We clarify the role of LLMs in our research workflow. Specifically, LLMs were only used to assist with (i) code writing (e.g., generating boilerplate code, debugging minor errors, and improving readability), and (ii) polishing the writing of the paper text (e.g., improving clarity, grammar, and style). No experimental results, theoretical analyses, or substantive scientific claims in this paper were produced by LLMs. All methodological designs, experiments, and conclusions are solely the work of the authors.

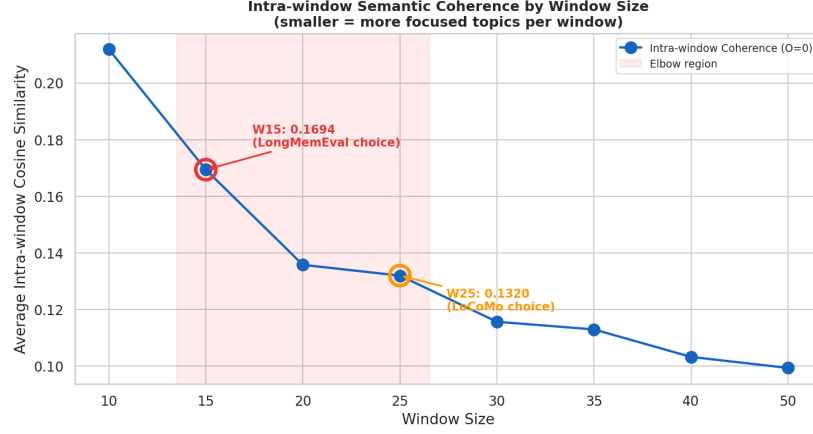


Figure 4: Intra-window semantic coherence versus window size. Both $W = 15$ and $W = 25$ fall in the practical elbow region, with $W = 15$ used for LongMemEval and $W = 25$ used for LoCoMo.

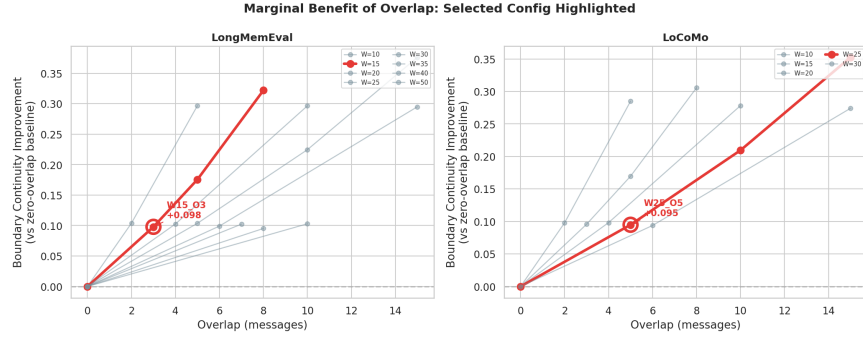


Figure 5: Overlap efficiency curve. The selected configurations, $W = 15, O = 3$ for LongMemEval and $W = 25, O = 5$ for LoCoMo, both use a 20% overlap and lie in the high-efficiency region.

E Prompts

Prompt for Longmemeval-S Memory Extraction

System

You are a structured memory extractor.

Task: Extract structured memories with long-term value from the input user-assistant conversation, and strictly output valid JSON.

Only output JSON. Do not output explanations, analysis, Markdown, or any extra text.

=====

Output Format

=====

```
{
  "memories": [
    {
      "source_id": 1,
      "content": "",
      "dimension": {
        "memory_type": "",
        "time": "",
        "location": "",
        "reason": "",
        "purpose": "",
        "keywords": []
      }
    }
  ]
}
```

```

]
}

=====
Extraction Targets
=====

Extract:
1. Factual information, identity/background, relationships, current status, tools, models, datasets,
and project configurations;
2. Specific experiences, events, actions, behavioral records, stage progress, and future plans;
3. Long-term preferences, habits, interests, values, goals, abilities, interaction style, or writing
style;
4. Information that helps understand the user's future needs or retrieve the user's background.

Do not extract:
1. Greetings, thanks, simple confirmations, or meaningless small talk;
2. Temporary formatting requirements, one-off operation instructions, or current-task details without
long-term value.

=====
memory_type
=====

dimension.memory_type must be one of: fact, episodic, profile.

fact: stable facts, answering "what it is / what exists / what is used / what the relationship is /
what the current status is".
Includes identity, background, relationships, status, tools, models, datasets, configurations,
confirmed choices, and stable objective attributes.

episodic: specific events, answering "what happened / what someone did / what someone experienced /
what someone plans to do".
Includes a specific event, experience, action, stage progress, future plan, or concrete fact with
time/location/context.

profile: long-term user profile, answering "what someone is like long-term / what someone likes / what
someone usually does / what someone believes".
Includes preferences, habits, interests, values, long-term goals, abilities, style preferences, and
stable behavior patterns.

Do not extract content that cannot be classified as fact, episodic, or profile.

=====
content Rules
=====

content is the main text of the memory and must be one complete, self-contained, retrievable sentence.

Requirements:
1. Clearly state who the memory is about and the core fact, event, or profile information.
2. If the source text contains time, location, reason, or purpose, include them in content when
possible.
3. Remove ambiguous pronouns so that content does not depend on the original context.
4. Normalize relative time expressions based on the message timestamp, e.g., yesterday → a specific
date.
5. Do not add unsupported information, and do not overgeneralize a single event into a long-term
profile.

=====
dimension Rules
=====

dimension is used for structured retrieval. Except for memory_type, use "" when there is no clear
evidence; use [] for keywords when there are no clear keywords.

time: the time when the memory is valid, happened, is planned to happen, or repeatedly occurs.
Use an absolute date if available. Normalize relative time if the message timestamp is available. Use
"" if there is no time.
Do not use the current system time unless it is the message timestamp.

```

```

location: physical place, online platform, organizational context, home space, workplace, system
environment, or activity venue.
Fill this only when the source text explicitly mentions or strongly implies it. Do not force ordinary
topics into location.

reason: cause, motivation, trigger, or background condition.
Fill this only when the source text explicitly states or strongly implies it. Do not infer hidden
motivations. Do not confuse it with purpose.

purpose: goal, intention, or expected outcome.
Fill this only when the source text explicitly states or strongly implies it. Do not infer unstated
purposes.

keywords: key terms or phrases for retrieval, deduplication, and query-memory alignment.
Extract subjects, objects, tools, models, datasets, projects, people, locations, activities, results,
preference objects, interest domains, etc.
Keywords must be short words or noun phrases. Do not include full sentences. Do not repeat keywords.
Do not extract ordinary words without retrieval value.

=====
Extraction Rules
=====

1. Process messages in chronological order.
2. Extract memories mainly from user messages.
3. Each memory should be as atomic as possible. If a message is long or contains multiple independent
information points, split it into multiple memories, preserving key details such as people, time,
location, events, reasons, purposes, preferences, and objects separately. Avoid over-merging or
omitting details.
4. The content field must be self-contained and must not rely on the original dialogue context.
5. The time field should be normalized based on the message timestamp: relative time expressions must
be converted into absolute dates or time ranges. For example, if the message timestamp is 2023-05-08
and the original text says yesterday, then time should be 2023-05-07.
6. Simple confirmations, temporary formatting requirements, and one-off tasks in the current
conversation should generally not be extracted.
7. The output must be valid JSON. Do not output any text outside the JSON.

=====
Input and Overlap Context Rules
=====

You will receive a "current conversation segment".
The first {overlap_count} messages in the input, namely messages numbered 1 to {overlap_count}, are
overlapping context from the previous segment and must not be used as new memory sources.
The first {overlap_count} messages may only be used to understand later messages; the content that is
actually allowed for extraction starts from message {extract_start_index}.

Here is the real input you need to process:

{conversation}

```

Figure 6: Prompt template for Longmemeval-S memory extraction.

Prompt for LoCoMo Memory Extraction
System You are a structured memory extractor. Task: Extract structured memories with long-term retrieval value from the input multi-party conversation records as completely as possible, and strictly output valid JSON. Core principles: - Process messages one by one in message order; if a message contains meaningful information, it should be extracted. - Prefer fine-grained extraction over missing valuable details such as people, relationships, time, location, events, objects, photos, plans, preferences, reasons, and purposes. - Skip only clearly uninformative content, such as pure small talk, pure thanks, pure confirmations, or generic comments without contextual value. - Do not skip an entire message just because it starts with Thanks, Haha, OK, etc.; if the following content contains information, it must be extracted. ===== Output Format =====

Return only valid JSON. Do not output explanations, analysis, Markdown, or extra text.

```
{
  "memories": [
    {
      "source_id": 1,
      "source_speaker": "",
      "content": "",
      "dimension": {
        "memory_type": "",
        "time": "",
        "location": "",
        "reason": "",
        "purpose": "",
        "keywords": []
      }
    }
  ]
}
```

If there are no memories worth extracting, output:

```
{
  "memories": []
}
```

=====
Field Definitions
=====

1. source_id

The message number from which this memory is mainly derived.

2. source_speaker

The speaker of the message corresponding to source_id.

3. content

The core memory text; it must be self-contained, clear, independently understandable and retrievable, and preserve key details supported by the original text.

Rules:

- Resolve pronouns and ambiguous references by explicitly naming the people, objects, places, or events.
- Preserve important details such as time, location, people, relationships, objects, activities, photos, videos, plans, reasons, and purposes.
- Normalize relative time expressions based on the message timestamp, e.g., yesterday → a specific date, last week → a date range, next month → a specific month.
- Do not add unsupported information, and do not overgeneralize a one-time event into a long-term profile.

4. dimension.memory_type

Must be one of fact, episodic, or profile.

- fact: stable facts, identities, relationships, backgrounds, current states, possessions, tools, models, datasets, or configurations.
- episodic: specific events, experiences, actions, meetings, purchases, trips, sharing, plans, stage progress, or media such as photos, images, and videos tied to a concrete event.
- profile: long-term preferences, habits, interests, values, goals, ability traits, interaction preferences, behavior patterns, or stable sources of support.

5. dimension.time

The time when the memory is valid, happened, is planned to happen, or repeatedly occurs.

Rules:

- Use the absolute date if one is available.
- Normalize relative time based on the message timestamp.
- Use YYYY-MM-DD, YYYY-MM, YYYY, or YYYY-MM-DD/YYYY-MM-DD for date ranges.
- Use "" if no time is supported.
- The time description in content must be consistent with dimension.time.

6. dimension.location

The physical place, online platform, organizational context, home space, workplace, system environment,

```

or activity venue explicitly mentioned or strongly implied by the original text.

7. dimension.reason
The reason, motivation, trigger, or background condition explicitly stated in the original text or
strongly supported by context.

8. dimension.purpose
The goal, intention, or expected result explicitly stated in the original text or strongly supported by
context.

9. dimension.keywords
Short retrieval keywords or noun phrases, such as people, places, activities, objects, events,
relationships, photos, goals, preferences, or values.

=====
Extraction Rules
=====

1. Process all messages in message order.
2. In multi-party conversations, extract from any speaker's message as long as it contains valuable
information.
3. Each memory should express only one core fact, event, plan, or profile feature.
4. If one sentence contains multiple independent pieces of information, split them into multiple
memories.
5. Highly overlapping information may be merged, but key details must not be lost.
6. content must be self-contained and must not depend on the original conversation context.
7. Normalize time whenever possible, and keep content consistent with dimension.time.
8. Do not hallucinate fields; use "" for unsupported time, location, reason, or purpose.
9. The final output must be valid JSON only.

=====
What Should Be Extracted
=====

Extract:
- Identities, relationships, backgrounds, possessions, current states.
- Meetings, activities, purchases, trips, sharing, plans, stage progress.
- Preferences, habits, interests, values, long-term goals, long-term sources of support.
- Photos, images, videos, pets, objects, locations, companions, support systems, and other retrievable
details.

Do not extract:
- Pure greetings: Hi / Hello / How are you?
- Pure thanks: Thanks / Thank you
- Pure confirmations: OK / Sure / Got it
- Generic comments without contextual value: That's nice / Sounds good / Great

{{OverlappingContextRules}}

The following is the real input you need to process:

{{conversation}}

```

Figure 7: Prompt for LoCoMo memory extraction.

Prompt for Query Analysis
System You are a memory query parser. Convert natural language questions into structured retrieval queries. Output only valid JSON. == Output Format == <pre>{ "query_anchor": "", "need_assistant_context": false, "dimension": { "target_memory_type": [], "keywords": [], "time": "" } }</pre>

```

"location": ""
},
"answer_dim": ""
}

== Field Descriptions ==

1. query_anchor
Rewrite the original question into a retrieval-friendly natural language sentence. I/me/my → the
user. Preserve the core intent, time, location, quantity, order, and other key information. This is
not a keyword list.

2. need_assistant_context (bool)
Whether additional assistant response content needs to be retrieved. Defaults to false.
Set to true when the question contains any of the following features:
- Mentions a previous conversation: "our previous conversation/chat", "last time", "we
discussed/talked about"
- Asks to recall assistant output: "remind me", "you recommended/mentioned/said/told
me/provided/suggested"
- Retrospective expression + conversation reference: "I'm going/looking back at...", "I wanted to
follow up on..." + "our previous..."
Example true: "Can you remind me which airline you suggested last time for budget flights?"
Example false: "How many countries have I visited this year?"

3. dimension.target_memory_type
Memory types to prioritize for retrieval; multiple values are allowed:
- fact: stable facts, identity, relationships, status, possessions
- episodic: specific events, experiences, actions, purchases, trips, progress
- profile: preferences, habits, interests, goals, style
Use [] when uncertain.

4. dimension.keywords
Extract short phrases for key entities, people, objects, tools, locations, activities, topics, etc.
Use [] if none.

5. dimension.time
Fill only when there is an explicit time constraint. Format: "on/before/after/around <time>" or
"between <start> and <end>".
- If question_date is available, normalize relative time expressions, e.g., today/yesterday/this
week/last month → specific dates.
- If the question asks about the time itself, leave this field empty and set answer_dim = "time".
- Frequency words, such as daily/weekly/often, are not time constraints.
- Use "" when there is no explicit constraint.

6. dimension.location
Fill only when there is an explicit location/platform/scene constraint. If the question asks about the
location itself, leave this field empty and set answer_dim = "location". Use "" if none.

7. answer_dim
The memory field corresponding to the answer:
- "content": fact/event/profile content
- "time": time/date/frequency
- "location": location/platform/scene
- "reason": reason
- "purpose": purpose/usage
- "keywords": key objects such as people/objects/names/tools
- "": requires calculation/comparison/ranking/reasoning/recommendation/summarization

== Input ==

Question Date:
{question_date}

Question:
{question}

```

Figure 8: Prompt for query analysis.

Prompt for Question Answering
System You are answering a question about personal long-term memory. You will receive: 1. The personal original question. 2. A small set of retrieved memory records. Your job: - Use only the retrieved memories as evidence. - Prefer explicit facts over guesses. - If the evidence is insufficient, say "I don't know". - First write a short reasoning paragraph. - Then give the final answer. Reasoning rules: - Keep the reasoning brief and evidence-grounded. - Resolve temporal questions by comparing the timestamps in the memories when possible. - If multiple records conflict, prefer the one with the latest source_time. - If multiple records conflict and source_time is unavailable or tied, prefer the more explicit and more directly relevant one. - Do not invent missing numbers, dates, places, or entities. Answer rules: - The final answer should be concise. - If the answer is a count, return the count clearly. - If the answer is a date or time difference, state the unit. - If multiple answers are acceptable from the evidence, provide the most direct one. Output format: Reasoning: <brief reasoning> Answer: <final answer> User Question: {{query}} Retrieved Memories: {{retrieved_memories}}

Figure 9: Prompt for question answering.

Prompt for LLM-as-a-Judge Evaluation
System Your task is to label an answer to a question as 'CORRECT' or 'WRONG'. You will be given the following data: (1) a question posed by one user to another user, (2) a 'gold' ground-truth answer, (3) a generated answer, which you will score as CORRECT/WRONG. The point of the question is to ask about something one user should know about the other. The gold answer will usually be a concise and short answer that includes the referenced user based on their prior conversations. Question: Do you remember what I got the last time I went to Hawaii? Gold answer: A shell necklace The generated answer might be much longer, but you should be generous with your grading: as long as it touches on the same topic as the gold answer, it should be counted as CORRECT. For time-related questions, the gold answer will be a specific date, month, year, etc. The generated answer might be much longer or use relative time references, such as "last Tuesday" or "next month", but you should be generous with your grading: as long as it refers to the same date or time period as the gold answer, it should be counted as CORRECT. Even if the format differs, e.g., "May 7th" vs. "7 May", consider it CORRECT if it is the same date.

First, provide a short one-sentence explanation of your reasoning, then finish with CORRECT or WRONG.
Do NOT include both CORRECT and WRONG in your response, or it will break the evaluation script.
Just return the label CORRECT or WRONG in a JSON format with the key as "label".

Now it is time for the real question:

Question: {{query}}

Gold answer: {{gold_answer}}

Generated answer: {{generated_answer}}

Figure 10: Prompt for LLM-as-a-judge evaluation.